Edge cases (a template with the wrong variables) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `prompt_template_practice.py`: a broken template can fail in two very different places — while building the prompt, or only once the API call goes out — and knowing which one changes how fast you can debug it. Seeing both a missing and an extra variable here, once, on purpose, means you recognize the failure instantly later. **If not:** the Build Task's `prompts.py` requirement to "fail the same clear way" would ask you to guarantee behavior you'd never actually observed happening.

Solution

```
# prompt_template_practice.py
from langchain_core.prompts import ChatPromptTemplate

def demonstrate_missing_variable(prompt: ChatPromptTemplate) -> None:
    # why: catching KeyError specifically documents exactly what failure
    # is expected here — a broad except would hide a genuinely different bug.
    try:
        prompt.invoke({})
    except KeyError:
        # how: the error message names the missing slot ('question') —
        # print it with {e} when you need to see which one
        print("Missing variable -> KeyError, before any API call")

def demonstrate_extra_variable(prompt: ChatPromptTemplate) -> None:
    # when: an unused key in the input dict is silently ignored, not an
    # error — worth confirming, since it's easy to assume the opposite.
    result = prompt.invoke({"question": "What is LCEL?", "extra": "unused"})
    # how: the filled prompt is a list of messages; [0] is the only one
    print(f"Extra variable -> no error: {result.messages[0].content}")

def main() -> None:
    prompt = ChatPromptTemplate.from_template("Answer: {question}")
    demonstrate_missing_variable(prompt)
    demonstrate_extra_variable(prompt)
```

```
if __name__ == "__main__":
    main()
```

Expected output:

```
Missing variable -> KeyError, before any API call
Extra variable -> no error: Answer: What is LCEL?
```

Which one should you write? Intermediate's specific-`KeyError` catch is what this document's Build Task needs — a clear error while building the prompt, not silently left blank. A stricter check — reading `prompt.input_variables` directly and validating both missing *and* extra (typo'd) keys before ever calling `.invoke()` — is worth adding anywhere you're building prompts from user input or config, where a typo'd key is a real, easy mistake; not required for this exercise or the Build Task.